University of Asia Pacific

DEPARTMENT OF CSE

COURSE CODE : 402
COURSE TITLE : Operating System Lab

DATE OF SUBMISSION : 17/10/2026

SUBMITTED BY:

NAME : Fairoj Salim
STUDENT ID : 23101165
SECTION : C2
LEVEL : 4th Year 1st Semester

SUBMITTED TO:

Atia Rahman Orthi
LECTURER
Department of CSE, UAP

PROBLEM STATEMENT

Given the following reference string and frame size, implement the FIFO (First In First Out) Page Replacement Algorithm using Python.

Parameter	Value
Reference String	1, 2, 3, 4, 2, 3, 1
Frame Size	3

The program should check whether each page is already present in the available frames. If the page is present, count it as a page hit; otherwise, count it as a page fault and load the page into a frame. When all frames are full, replace the page that entered the frames first. Finally, calculate the total number of page hits, page faults, hit ratio, and fault ratio.

SOURCE CODE

```
pages = [1, 2, 3, 4, 2, 3, 1]
frames = []
size = 3
hit = 0
fault = 0

for page in pages:
    if page in frames:
        hit += 1
        print(page, frames, "Hit")
    else:
        fault += 1

        if len(frames) == size:
            frames.pop(0)

        frames.append(page)
        print(page, frames, "Fault")

print("\nTotal Page Hits:", hit)
print("Total Page Faults:", fault)
print("Hit Ratio:", round(hit / len(pages), 2))
print("Fault Ratio:", round(fault / len(pages), 2))
```

OUTPUT



DISCUSSION

This program implements the FIFO (First In First Out) Page Replacement Algorithm, which manages a fixed number of memory frames by always evicting the page that has been sitting in memory the longest, irrespective of how often or how recently it has been accessed.

For the reference string 1, 2, 3, 4, 2, 3, 1 with a frame size of 3, the first three references — pages 1, 2, and 3 — fill the empty frames one by one, producing three page faults since none of them were in memory yet. When page 4 is referenced next, all three frames are already occupied, so the algorithm removes page 1, the page that has stayed in memory the longest, and loads page 4 in its place. At this point the frames hold {2, 3, 4}.

The following two references, page 2 and page 3, are both already present in the current frame set, so each is counted as a page hit without triggering any replacement. The final reference, page 1, is no longer in memory — it was evicted earlier — so it causes another page fault, and this time page 2 (now the oldest resident of the frames) is replaced.

Overall, the simulation produced 2 page hits and 5 page faults out of 7 total references, giving a Hit Ratio of 0.29 (about 28.57%) and a Fault Ratio of 0.71 (about 71.43%). These numbers illustrate the core behaviour of FIFO: replacement decisions are made purely on arrival order, with no regard for how useful or frequently used a page actually is — which is also why FIFO is considered one of the simplest, but not necessarily the most efficient, page replacement strategies.

CONCLUSION

The FIFO Page Replacement Algorithm was implemented in Python for a frame size of 3 and the reference string 1, 2, 3, 4, 2, 3, 1, and its behaviour was traced step by step to confirm how pages enter and leave memory.

- **Result:** The run produced 2 page hits and 5 page faults, giving a Hit Ratio of 0.29 (28.57%) and a Fault Ratio of 0.71 (71.43%).

- **How it decides:** Whenever a new page needs to be loaded into a full frame set, FIFO always discards the page that arrived earliest, following a simple queue-like order.
- **Strength:** The algorithm is straightforward to implement and reason about, since it only needs to track the order in which pages entered memory.
- **Limitation:** Because it ignores how recently or frequently a page is actually used, FIFO can evict a page that is still needed, which is why its hit ratio can vary considerably depending on the reference string and frame size used.